

Introduction to C

History of C language:

- **1960:** ALGOL introduced as the first block-structured programming language, influencing future languages.
- **1967:** BCPL (Basic Combined Programming Language) developed by Martin Richards, serving as a precursor to C.
- **1970:** B language created by Ken Thompson, a simplified version of BCPL, primarily used in academic settings.
- **1972:** C language developed by Dennis Ritchie at Bell Labs as a system programming language for the Unix operating system, enhancing B with data typing and other features.
- **1973:** The UNIX kernel was rewritten in C, demonstrating its capabilities and efficiency.
- **1978:** Publication of "**The C Programming Language**" by Brian Kernighan and Dennis Ritchie, commonly referred to as K&R C, establishing a standard reference for the language.
- **1983:** The American National Standards Institute (ANSI) formed the X3J11 committee to **standardize C**.
- **1989:** ANSI C (also known as C89) was published, introducing standardized syntax and libraries, promoting portability across different systems.
- **1990:** Minor updates led to the formal adoption of **C90**, which included slight modifications to ANSI C.
- **1999:** The C99 standard was released, adding features such as variable length arrays, new data types (like bool), and inline functions, enhancing the language's functionality and flexibility.

This timeline highlights the evolution of the C programming language from its inception to the establishment of significant standards up to C99.

Features of C language:

C is a widely-used programming language known for its simplicity, efficiency, and flexibility.

Here are its key features:

1. **Simple and Structured:** C has a relatively simple syntax that is easy to understand and write. Its structured programming nature allows breaking down a complex problem into smaller, manageable modules or functions. **This makes code easier to read, debug, and maintain.**
2. **Portable:** Programs written in C can be compiled on one system and run on another with minimal changes. This portability is due to the availability of C compilers for almost all hardware platforms and operating systems, making C highly versatile.
3. **Low-Level Access:** C provides low-level programming capabilities through pointers and bitwise operators. This feature makes it possible to write programs that interact directly with hardware components, such as device drivers and operating systems.

4. **Efficient and Fast:** Since C is a middle-level language, it strikes a balance between high-level abstraction and low-level hardware access. Its compiled nature translates code directly into machine language, leading to faster execution compared to interpreted languages.
5. **Rich Library Support:** C comes with standard libraries like the Standard Input/Output Library (stdio.h), Math Library (math.h), and String Manipulation Library (string.h). These libraries provide ready-made functions for common tasks, reducing development time.
6. **Modularity:** Programs in C can be divided into functions or modules, allowing for better organization and reusability. Functions can be written once and used multiple times in the same program or even in other programs, promoting code efficiency.
7. **Extensibility:** New features can be added to existing programs without significant restructuring. By defining additional functions or using third-party libraries, developers can expand a program's capabilities seamlessly.
8. **Wide Application:** C is used in a variety of fields, including operating system development (e.g., UNIX, Linux, and Windows), embedded systems, game development, compiler design, and high-performance applications. Its versatility and efficiency make it a preferred choice for system-level programming.

C's combination of simplicity, power, and flexibility makes it a foundational language that remains relevant decades after its creation. It is often considered a stepping stone for learning other programming languages and systems.

C as a middle-level language:

C is referred to as a middle-level language because it combines features of both high-level and low-level languages. Here's why:

1. High-Level Language Features:

- **Abstraction:** C provides high-level constructs such as functions, loops, and conditionals that make it easier to write, read, and understand code compared to assembly language.
- **Portability:** Programs written in C can run on different machines with minimal changes, a hallmark of high-level languages.
- **Built-in Libraries:** C has standard libraries for handling input/output, math, string manipulation, and more, simplifying common programming tasks.

2. Low-Level Language Features:

- **Direct Hardware Access:** C allows direct interaction with hardware through pointers and memory manipulation, which is typically associated with low-level programming.
- **Efficient Code Execution:** Since C compiles directly into machine code, it offers performance similar to assembly language.
- **Bitwise Operators:** C includes operators for bit-level operations, enabling precise control over hardware and memory.

C is versatile enough to be used for high-level application development (e.g., desktop applications) and low-level system programming (e.g., embedded systems). This dual nature is why it is called a "middle-level language".

C is called a middle-level language because it provides the tools and abstractions of high-level languages while allowing low-level access and efficient interaction with hardware, making it a bridge between the two.

C Character set:

The C programming language utilizes a character set that encompasses all valid characters used to construct programs, including letters, digits, special symbols, and whitespace

Categories of the C Character Set

1. Alphabets

C supports both uppercase and lowercase English letters, totaling 52 characters:

Uppercase Letters: A to Z (ASCII values 65 to 90)

Lowercase Letters: a to z (ASCII values 97 to 122)

2. Digits

C includes the ten digits from 0 to 9, which are represented by their ASCII values:

0 (ASCII 48) to 9 (ASCII 57)

3. Special Characters

C allows a variety of special symbols used for operations, conditions, and formatting. Some examples include:

Arithmetic operators: +, -, *, /

Logical operators: &&, ||, !

Punctuation: ;, :, ., ,, ?, etc.

4. Whitespace Characters

Whitespace characters are critical for formatting in C. They include:

Space (' '), Carriage Return ('\r'), New Line ('\n'), Tab ('\t'), and others.

C tokens

C tokens are the smallest individual elements of a C program that have specific meanings to the compiler. They serve as the building blocks for constructing C programs, and understanding them is essential for effective programming. ***The C language categorizes tokens into six primary types:***

1. ***Keywords***
2. ***Identifiers***
3. ***Constants***
4. ***Operators***
5. ***Special Characters***
6. ***Strings***

Keywords in C

Keywords are predefined, reserved words in the C language that have special meanings and cannot be used as identifiers (such as variable names). They are integral to the syntax of C and help define the control flow, data types, and other essential elements of a program. **As per the C99 standard, there are 32 keywords. These keywords are case-sensitive, meaning that variations like uppercase or lowercase letters represent different entities.**

Keywords			
auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
continue	for	signed	void
do	if	static	while
default	goto	sizeof	volatile
const	float	short	unsigned

Identifiers in C

Identifiers are names given to various program elements such as variables, functions, arrays, etc., defined by the programmer. Unlike keywords, identifiers can be chosen freely but must adhere to specific rules:

Rules for Identifiers

- **Starting Character:** Must begin with a letter (A-Z, a-z) or an underscore (_).
- **Subsequent Characters:** Can include letters, digits (0-9), and underscores.
- **Case Sensitivity:** Identifiers are case-sensitive; for example, myVar, MyVar, and MYVAR are different identifiers.
- **Length:** There is no strict limit on length, but it is advisable to keep them reasonably short for readability.
- **No Reserved Keywords:** Identifiers cannot be the same as any of the reserved keywords.

Valid identifiers: myVariable, _temp, count1, calculateSum

Invalid identifiers: 1stVariable (cannot start with a digit), int (cannot use reserved keywords)

Data types

In C programming, data types are essential for defining the type of data that can be stored in a variable. They determine the size and layout of the variable's memory, the range of values it can hold, and the operations that can be performed on it.

1. Primary data types

The primary data types in the C programming language are the basic data types. All the primary data types are already defined in the system. Primary data types are also called as Built-In data types. The following are the primary data types in c programming language.

- ***Integer data type***
- ***Floating Point data type***
- ***Character data type***

a. Integer Data type

The integer data type is a set of whole numbers. any integer variable does not hold the decimal value. We use the keyword int to declare the variables and to specify the return type of a function.

Type	Size (bytes)	Range	Specifier
int (signed short int)	2	-32768 to +32767	%d
short int (signed short int)	2	-32768 to +32767	%d
long int (signed long int)	4	-2,147,483,648 to +2,147,483,647	%d
unsigned int (unsigned short int)	2	0 to 65535	%u
unsigned long int	4	0 to 4,294,967,295	%u

b. Floating Point data types

Floating-point data types are a set of numbers with the decimal value. Every floating-point value must contain the decimal value. The floating-point data type has two variants, float and double.

Type	Size (bytes)	Range	Specifier
float	4	1.2E - 38 to 3.4E + 38	%f
double	8	2.3E-308 to 1.7E+308	%d
long double	10	3.4E-4932 to 1.1E+4932	%d

c. Character data type

The character data type is a set of characters enclosed in single quotations. The following table provides complete details about the character data type.

Type	Size (bytes)	Range	Specifier
char (signed char)	1	-128 to +127	%c
unsigned char	1	0 to 255	%c

2. Derived Data Types

Derived data types are constructed from the basic data types and include:

- **Arrays:** A collection of elements of the same type.
- **Pointers:** Variables that store memory addresses.
- **Structures:** A user-defined data type that groups different types together.
- **Unions:** Similar to structures but share the same memory location for all members.

3. Enumerated Data Types

Enumerated data types (enums) allow programmers to define variables that can hold a set of predefined constants, enhancing code readability.

```
enum Day { Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday};
```

4. Void Data Type

The void type signifies that no value is available or that a function does not return a value. It is often used in function declarations.

```
void displayMessage()  
{  
    printf("Hello, World!");  
}
```

Constants and variables

Constants

Constants are immutable values that cannot be modified once they have been declared. They are used to represent fixed values throughout a program. There are two primary ways to define constants in C.

1. Using the *const* Keyword

The *const* qualifier is used to declare a variable as a constant. Once defined, its value cannot be changed during the program's execution.

Syntax: `const data_type variable_name = value;`

Example:

```
const int MAX_USERS = 100; // MAX_USERS cannot be changed
```

2. using the #define Preprocessor Directive

The #define directive allows you to create symbolic constants, which are replaced by their values at compile time.

Syntax: #define CONSTANT_NAME value

Example:

```
#define PI 3.14159 // PI is replaced with 3.14159 in the code
```

Important Properties of Constants

- **Initialization:** Constants must be initialized at the time of declaration; otherwise, they will contain garbage values.
- **Immutability:** Once assigned, the value of a constant cannot be altered.
- **Scope:** Constants defined with #define have a global scope, while those defined with const can have local or global scope depending on where they are declared.

Variables

Variables are named storage locations in memory that can hold data that may change during program execution. Unlike constants, variables can be modified after their initial declaration.

Declaration of Variables

To declare a variable in C, you specify the data type followed by the variable name.

Syntax: data_type variable_name;

Example: int age; // Declares an integer variable named age

To declare a multiple variables in C, you specify the data type followed by the variables names.

Syntax: data_type variable_name1, variable_name2, variable_name3, variable_name4;

Example: int Number_1, Number2, Sum; // Declares an integer variables Number1, Number2 and Sum

Initialization of Variables

Variables can also be initialized at the time of declaration.

Example: int score = 50; // Declares and initializes score to 50

Types of Variables

1. Local Variables

Local variables are declared within a function or a block and can only be accessed within that function or block. They are created when the function is called and destroyed when the function exits.

Example:

```
#include <stdio.h>
```

```

void myFunction() {
    int localVar = 10; // Local variable
    printf("Local Variable: %d\n", localVar);
}

int main() {
    myFunction();
    printf("%d", localVar); // This would cause an error
    return 0; }

```

2. Global Variables

Global variables are declared outside of all functions and can be accessed by any function within the same file or other files if declared with the extern keyword. They exist for the lifetime of the program.

Example:

```

#include <stdio.h>

int globalVar = 20; // Global variable

void myFunction() {
    printf("Global Variable: %d\n", globalVar);}

int main() {
    myFunction();
    printf("Global Variable in main: %d\n", globalVar);
    return 0;}

```

3. Static Variables

Static variables retain their value between function calls and are initialized only once. They can be either local or global but have a lifetime that extends throughout the program's execution.

Example of a Static Local Variable:

```

#include <stdio.h>

void myFunction() {
    static int count = 0; // Static variable
    count++;
    printf("Count: %d\n", count);}

int main() {
    myFunction(); // Output: Count: 1
    myFunction(); // Output: Count: 2
    return 0; }

```

4. Automatic Variables

Automatic variables are local variables that are automatically allocated when the block is entered and deallocated when it is exited. By default, all local variables are automatic unless specified otherwise (e.g., with static).

Example:

```
#include <stdio.h>

void myFunction() {
    int autoVar = 5; // Automatic variable
    printf("Automatic Variable: %d\n", autoVar);}

int main() {
    myFunction();
    return 0;}
```

Formatted input and output functions:

Formatted input and output functions in C are essential for managing how data is read from users and displayed on the screen. The two primary functions used for formatted I/O in C are **printf()** for output and **scanf()** for input. Here's a detailed look at these functions, their syntax, and usage.

Formatted Output: printf()

The printf() function is used to display formatted output to the console. It allows programmers to specify the format in which data should be presented using format specifiers.

Syntax: printf("format_specifier", value1, value2, ...);

Example: printf("The total sum is:- %d", sum);

Common Format Specifiers

- %d: Used for signed integers.
- %u: Used for unsigned integers.
- %f: Used for floating-point numbers.
- %c: Used for single characters.
- %s: Used for strings (character arrays).
- %lf: Used for double precision floating-point numbers.
- %ld: Used for long integers.

Formatted Input: scanf()

The scanf() function is used to read formatted input from the user. It allows you to specify the expected format of the input data.

Syntax: scanf("format_specifier", &variable1, &variable2, ...);

Example: scanf("%d%d", &num1,&num2);

Operators in C:

C programming language provides a rich set of operators that allow developers to perform various operations on data. These operators can be classified into several categories based on their functionality:

1. Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations.

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b
++	Increment	++a or a++
--	Decrement	--a or a--

2. Relational Operators

Relational operators are used to compare two values and return a Boolean result (true or false).

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
<	Less than	a < b
>	Greater than	a > b
<=	Less than or equal to	a <= b
>=	Greater than or equal to	a >= b

3. Logical Operators

Logical operators are used to combine conditional statements.

Operator	Description	Example
&&	Logical AND	(a && b)
	Logical OR	(Expression1 Expression2)
!	Logical NOT	!a

4. Bitwise Operators

Bitwise operators perform operations on bits and are useful for low-level programming.

Operator	Description	Example
&	Bitwise AND	a & b
	Bitwise OR	a b
^	Bitwise XOR (exclusive OR)	a ^ b
~	Bitwise NOT (complement)	~a
<<	Left shift	a << 1
>>	Right shift	a >> 1

5. Assignment Operators

Assignment operators are used to assign values to variables, often in combination with arithmetic operations.

Operator	Description	Example
=	Simple assignment	a = b;
+=	Add and assign	a += b; // a = a + b;
-=	Subtract and assign	a -= b; // a = a - b;
*=	Multiply and assign	a *= b; // a = a * b;

Operator	Description	Example
/=	Divide and assign	a /= b; // a = a / b;
%=	Modulus and assign	a %= b; // a = a % b;

6. Conditional (Ternary) Operator

The conditional operator is a shorthand for the if-else statement, taking three operands.

Syntax: condition ? expression_if_true : expression_if_false;

Example: int max = (a > b) ? a : b;

7. Miscellaneous Operators

These include operators that do not fit into the above categories, such as:

Sizeof Operator: Returns the size of a data type or variable.

Example: int size = sizeof(int); // Returns the size of int type

Comma Operator: Allows two expressions to be evaluated in sequence.

Example: int x = (y = 3, y + 2); // y is assigned 3, then x is assigned y + 2

Operators in C can be also categorized based on the number of operands they require as followings:

- **Unary Operators:** Operate on one operand (e.g., increment, decrement).
- **Binary Operators:** Operate on two operands (e.g., arithmetic, relational).
- **Ternary Operators:** Operate on three operands (e.g., conditional operator).

Operators in C

	Operators	Type
Unary Operator →	++, --	Unary Operator
Binary Operator {	+, -, *, /, %	Arithmetic Operator
	<, <=, >, >=, ==, !=	Relational Operator
	&&, , !	Logical Operator
	&, , <<, >>, ~, ^	Bitwise Operator
	=, +=, -=, *=, /=, %=	Assignment Operator
Ternary Operator →	?:	Ternary or Conditional Operator